

LAB 10

MUSIC MANAGEMENT & REVIEW SYSTEM

You are required to develop a backend application for managing **music tracks** and collecting **listener reviews**.

Users can register, log in, log out, browse songs, submit ratings, and perform management operations.

The system provides **RESTful JSON APIs** and an **EJS-rendered HTML view**.

I. Database Structure (MongoDB)

System uses three collections: **Users, Songs, Reviews**

1 Users Collection

Field	Type	Description
_id	ObjectId	Unique identifier
username	String	Unique login name
password	String	Hashed using bcrypt
role	String	user / admin
createdAt	Date	Account creation time

Example:

```
{
  "username": "alice",
  "password": "$2b$10$hashed",
  "role": "user",
  "createdAt": "2026-03-10T08:30:00Z"
}
```

2 Songs Collection

Field	Type	Description
_id	ObjectId	Song ID
title	String	Song title
artist	String	Singer/Band
genre	String	Music genre
releaseYear	Number	Year released
duration	Number	Seconds
album	String	Album name
coverUrl	String	Cover image
averageRating	Number	Calculated from reviews
createdAt	Date	Created time

3 Reviews Collection

Field	Type	Description
id	ObjectId	Review ID
songId	ObjectId	Ref → Songs
userId	ObjectId	Ref → Users
rating	Number	Score (1–5)
comment	String	Feedback
createdAt	Date	Review time

II. Technical Requirements



Project Folder

MusicSys_StudentCode



Technologies

- Node.js
- Express.js
- MongoDB
- Mongoose
- EJS
- JWT
- bcrypt
- dotenv, cors

III. TASKS TO BE COMPLETED



Task 1 — Project Setup & Configuration

- Create database **MusicDB_2026**
- Create collections: Users, Songs, Reviews
- Import sample JSON
- Install dependencies
- Configure environment variables

.env

PORT=1234

MONGO_URI=mongodb://127.0.0.1:27017/MusicDB_2026

JWT_SECRET=music_secret_key



Task 2 — API: User Registration

POST /users/register

- Register new account
- Validate duplicate username
- Hash password using bcrypt
- Default role = user



Task 3 — API: User Login

POST /users/login

- Verify username & password
- Generate JWT token
- Token expires in 24h

Response

```
{ "token": "JWT_TOKEN" }
```



Task 4 — API: User Logout

POST /users/logout

- Client deletes token
- Optional: implement token blacklist

Response

```
{ "message": "Logged out successfully" }
```



Task 5 — API: Get All Songs (Public)

GET /songs

- Return full song list
- Sort by newest

✓ Task 6 — API: Get Song Details (Authenticated + Populate)

GET /songs/:id

Populate:

- All reviews of the song
- Username of reviewers

✓ Task 7 — API: Create New Song (Admin only)

POST /songs

✓ Task 8 — API: Update Song Information (Admin only)

PUT /songs/:id

✓ Task 9 — API: Delete Song (Admin only)

DELETE /songs/:id

✓ Task 10 — API: Add Review (Authenticated)

POST /reviews/:songId

- Each user can review a song once
- Auto-update averageRating

✓ Task 11 — API: Edit Review (Owner only)

PUT /reviews/:id

✓ Task 12 — API: Delete Review (Owner/Admin)

DELETE /reviews/:id

IV. STATISTICAL & ADVANCED QUERIES (Populate + Aggregation)

✓ Task 13 — My Review History

GET /reviews/myreviews

Populate song info:

- title
- artist
- genre

✓ Task 14 — Top Rated Songs

GET /songs/top-rated

- Sort by averageRating desc
- Limit top 10 songs

✓ Task 15 — Most Reviewed Songs

GET /songs/most-reviewed

Aggregation:

- Count reviews per song
- Sort descending

✓ Task 16 — Song Rating Statistics

GET /songs/:id/statistics

Return:

```
{
  "averageRating": 4.6,
  "totalReviews": 125,
  "ratingDistribution": {
```

```
    "5": 80,  
    "4": 30,  
    "3": 10,  
    "2": 3,  
    "1": 2  
  }  
}
```

 Task 17 — Admin: User Activity Report

GET /admin/user-activities

Populate:

- Username
- Number of reviews
- List of reviewed songs

V. Security Requirements

Protected routes require:

Authorization: Bearer <token>

Role-based middleware:

- user → review features
- admin → song management

DATABASE

File 1 — users.json

```
[
  {
    "_id": { "$oid": "66f100000000000000000001" },
    "username": "alice",
    "password": "$2b$10$hashedpassword1",
    "role": "user",
    "createdAt": { "$date": "2026-03-01T08:00:00Z" }
  },
  {
    "_id": { "$oid": "66f100000000000000000002" },
    "username": "bob",
    "password": "$2b$10$hashedpassword2",
    "role": "user",
    "createdAt": { "$date": "2026-03-02T09:00:00Z" }
  },
  {
    "_id": { "$oid": "66f100000000000000000003" },
    "username": "charlie",
    "password": "$2b$10$hashedpassword3",
    "role": "user",
    "createdAt": { "$date": "2026-03-03T10:00:00Z" }
  },
  {
    "_id": { "$oid": "66f100000000000000000004" },
    "username": "admin",
    "password": "$2b$10$hashedadmin",
    "role": "admin",
    "createdAt": { "$date": "2026-03-01T07:00:00Z" }
  }
]
```

File 2 — songs.json

```
[
  {
    "_id": { "$oid": "66f200000000000000000001" },
    "title": "Blinding Lights",
    "artist": "The Weeknd",
    "genre": "Synthwave",
    "releaseYear": 2019,
    "duration": 200,
    "album": "After Hours",
    "coverUrl": "/images/blindinglights.jpg",
    "averageRating": 4.8,
    "createdAt": { "$date": "2026-03-01T08:00:00Z" }
  },
  {
    "_id": { "$oid": "66f200000000000000000002" },
    "title": "Someone Like You",
    "artist": "Adele",
    "genre": "Soul",
    "releaseYear": 2011,
    "duration": 285,
    "album": "21",
    "coverUrl": "/images/someonelikeyou.jpg",
    "averageRating": 4.6,
    "createdAt": { "$date": "2026-03-02T08:00:00Z" }
  },
  {
    "_id": { "$oid": "66f200000000000000000003" },
    "title": "See You Again",

```

```

    "artist": "Wiz Khalifa ft. Charlie Puth",
    "genre": "Hip Hop",
    "releaseYear": 2015,
    "duration": 229,
    "album": "Furious 7",
    "coverUrl": "/images/seeyouagain.jpg",
    "averageRating": 4.7,
    "createdAt": { "$date": "2026-03-03T08:00:00Z" }
  },
  {
    "_id": { "$oid": "66f200000000000000000004" },
    "title": "Counting Stars",
    "artist": "OneRepublic",
    "genre": "Pop Rock",
    "releaseYear": 2013,
    "duration": 257,
    "album": "Native",
    "coverUrl": "/images/countingstars.jpg",
    "averageRating": 4.5,
    "createdAt": { "$date": "2026-03-04T08:00:00Z" }
  }
]

```

File 3 — reviews.json

```

[
  {
    "_id": { "$oid": "66f300000000000000000001" },
    "songId": { "$oid": "66f200000000000000000001" },
    "userId": { "$oid": "66f100000000000000000001" },
    "rating": 5,
    "comment": "Amazing retro vibe and powerful vocals!",
    "createdAt": { "$date": "2026-03-05T08:00:00Z" }
  },
  {
    "_id": { "$oid": "66f300000000000000000002" },
    "songId": { "$oid": "66f200000000000000000001" },
    "userId": { "$oid": "66f100000000000000000002" },
    "rating": 4,
    "comment": "Catchy and energetic song.",
    "createdAt": { "$date": "2026-03-05T09:00:00Z" }
  },
  {
    "_id": { "$oid": "66f300000000000000000003" },
    "songId": { "$oid": "66f200000000000000000002" },
    "userId": { "$oid": "66f100000000000000000001" },
    "rating": 5,
    "comment": "Emotional and beautifully performed.",
    "createdAt": { "$date": "2026-03-06T10:00:00Z" }
  },
  {
    "_id": { "$oid": "66f300000000000000000004" },
    "songId": { "$oid": "66f200000000000000000003" },
    "userId": { "$oid": "66f100000000000000000003" },
    "rating": 5,
    "comment": "Perfect tribute song, very touching.",
    "createdAt": { "$date": "2026-03-07T11:00:00Z" }
  },
  {
    "_id": { "$oid": "66f300000000000000000005" },
    "songId": { "$oid": "66f200000000000000000004" },
    "userId": { "$oid": "66f100000000000000000002" },
    "rating": 3,
    "comment": "Nice rhythm but a bit repetitive.",
    "createdAt": { "$date": "2026-03-08T12:00:00Z" }
  }
]

```

```
    "_id": { "$oid": "66f300000000000000000006" },
    "songId": { "$oid": "66f200000000000000000003" },
    "userId": { "$oid": "66f100000000000000000001" },
    "rating": 4,
    "comment": "Great collaboration and meaningful lyrics.",
    "createdAt": { "$date": "2026-03-08T13:00:00Z" }
  }
]
```



Hướng dẫn sinh viên import dữ liệu

Cách 1 — Dùng MongoDB Compass

1. Mở database `MusicDB_2026`
2. Chọn collection
3. **Add Data → Import JSON**
4. Chọn file tương ứng

Cách 2 — Dùng Mongo Shell

```
mongoimport --db MusicDB_2026 --collection users --file users.json --jsonArray
mongoimport --db MusicDB_2026 --collection songs --file songs.json --jsonArray
mongoimport --db MusicDB_2026 --collection reviews --file reviews.json --
jsonArray
```

TESTCASE

✅ Task 1: Project Setup

Project folder: MusicSys_StudentCode
Database name: MusicDB_2026
Collections: Users, Songs, Reviews
Server URL: http://localhost:1234

✅ Task 2: API – POST /users/register

📌 Endpoint

POST http://localhost:1234/users/register

📌 Postman

Tab **Body** → **raw** → **JSON**

```
{
  "username": "usera",
  "password": "123456"
}
```

📌 Expected Result

- User created
- Password hashed using bcrypt
- JSON success message returned

✅ Task 3: API – POST /users/login

📌 Endpoint

POST http://localhost:1234/users/login

📌 Postman

Tab **Body** → **raw** → **JSON**

```
{
  "username": "usera",
  "password": "123456"
}
```

📌 Expected Result

- JWT token returned using JWT

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

⚠️ **Note:** Save this token for next tasks.

✅ Task 4: API – POST /users/logout

📌 Endpoint

POST http://localhost:1234/users/logout

📌 Headers

Authorization: Bearer <token>

📌 Expected Result

```
{
  "message": "Logged out successfully"
}
```

✅ Task 5: API – GET /songs

📌 Endpoint

GET http://localhost:1234/songs

📌 Expected Result

JSON list of all songs:

```
[
  {
    "_id": "66f200000000000000000001",
```



```

    "title": "Blinding Lights",
    "artist": "The Weeknd",
    "genre": "Synthwave",
    "releaseYear": 2019,
    "averageRating": 4.8
  }
]

```

✅ Task 6: API – POST /reviews/:songId (Add Review)

📌 Endpoint

POST `http://localhost:1234/reviews/66f200000000000000000001`

📌 Headers

Authorization: Bearer <token>

📌 Body → raw → JSON

```

{
  "rating": 5,
  "comment": "This song is fantastic!"
}

```

📌 Expected Result

```

{
  "message": "Review added successfully"
}

```

✅ Task 7: API – GET /reviews/myreviews (Authenticated)

📌 Endpoint

GET `http://localhost:1234/reviews/myreviews`

📌 Authentication (Postman → Headers)

Key: Authorization

Value: Bearer <token>

📌 System Behavior

When logged-in user sends request:

- Verify JWT token
- Filter Reviews by `userId`
- Use **`populate('songId')`** to get song info

📌 Example JSON Result

```

{
  "user": "usera",
  "reviews": [
    {
      "song": {
        "_id": "66f200000000000000000001",
        "title": "Blinding Lights",
        "artist": "The Weeknd",
        "genre": "Synthwave"
      },
      "rating": 5,
      "comment": "This song is fantastic!",
      "createdAt": "2026-03-20T10:00:00Z"
    }
  ]
}

```

✅ Task 8: API – GET /songs/details/:id (Authenticated)

📌 Endpoint

GET `http://localhost:1234/songs/details/66f200000000000000000001`

Authentication Header

Authorization: Bearer <token>

System Behavior

- Verify JWT
- Find song by ID
- Nested populate reviews & usernames

```
Song.findById(req.params.id)
  .populate({
    path: 'reviews',
    populate: { path: 'userId', select: 'username' }
  })
```

Example JSON Result

```
{
  "_id": "66f200000000000000000001",
  "title": "Blinding Lights",
  "artist": "The Weeknd",
  "genre": "Synthwave",
  "releaseYear": 2019,
  "reviews": [
    {
      "rating": 5,
      "comment": "Fantastic!",
      "user": { "username": "alice" }
    },
    {
      "rating": 4,
      "comment": "Very catchy song.",
      "user": { "username": "bob" }
    }
  ]
}
```

Verification Checklist

- ✓ API returns correct song
- ✓ Reviews fully displayed
- ✓ Username populated correctly
- ✓ JWT required
- ✓ Correct linking: Songs ↔ Reviews ↔ Users

Task 9: API – PUT /reviews/:id (Edit Review)

PUT http://localhost:1234/reviews/66f300000000000000000001

Headers:

Authorization: Bearer <token>

Body:

```
{
  "rating": 4,
  "comment": "Updated review after listening again."
}
```

Expected:

Review updated successfully

Task 10: API – DELETE /reviews/:id

DELETE http://localhost:1234/reviews/66f300000000000000000001

Headers:

Authorization: Bearer <token>

Expected:

Review deleted successfully

✓ Task 11: API – GET /songs/top-rated

GET `http://localhost:1234/songs/top-rated`

Expected:

- Sorted by rating DESC
- Top 10 songs

✓ Task 12: API – GET /songs/most-reviewed

GET `http://localhost:1234/songs/most-reviewed`

Expected:

- Aggregation count reviews
- Sort descending

✓ Task 13: API – GET /songs/:id/statistics

GET `http://localhost:1234/songs/66f2000000000000000001/statistics`

Expected:

```
{
  "averageRating": 4.8,
  "totalReviews": 25,
  "ratingDistribution": {
    "5": 15,
    "4": 7,
    "3": 2,
    "2": 1,
    "1": 0
  }
}
```